CS 432 Distributed Systems	Lab (5)

Section Content:

- **JSON: JavaScript Object Notation**
- **SHA-256: Secure Hash Algorithm**
- **QoS: Quality of Service**
- **Interfaces in Distributed Systems**
- **Heartbeat**

The story: Node-A needs to print an important report

Imagine a university has a distributed system:

- Node-A: The computer used by an employee who wants to print a report.
- Node-B: A server that provides the printing service.
- The printer: The device that prints the report.

Initially, everything seems simple. Node-A sends a request to Node-B, and Node-B prints the report. But five different problems appear.

Problem 1: Node-A and Node-B cannot understand the request

Node-A is programmed in **Java**, but **Node-B** is programmed in **Python**. **Node-A sends a request** like this: **Print report.pdf, 5 pages**
Node-B expects a **different format**, **so it doesn't know how to interpret the request.**

Solution use JSON (JavaScript Object Notation)

What is JSON?

JSON is a simple **text format** for sending structured data from one program (or node) to another.

Why do we need it? Node-A may be written in Java and Node-B in Python. They need a **common language** for their data. JSON is that language. Humans can read it, and every programming language can understand it.

Key-value pairs and data types

JSON stores data as **key : value** pairs inside curly brackets { }.

Example: Node-A sends a message to Node-B



- nodeId tells who the **sender** is.
- service tells which service is **requested**.
- pages is a **number**.
- success is a **Boolean** value.

Heterogeneity means that distributed system nodes are different, while **interoperability** means they **can work together** despite these differences. **JSON** helps **achieve interoperability** by providing a common format for exchanging data.

Program1: Heterogeneity and Interoperability Using JSON

This program demonstrates Heterogeneity and Interoperability in distributed systems. A node creates a message and converts it into JSON (JavaScript Object Notation), which is a platform-independent data format. Because JSON is text-based and standardized, nodes written in Python, C++, Script, or any other language can easily parse and understand the message.

Code

```
package interoperabilitydemo;
public class InteroperabilityDemo {
    public static void main(String[] args) {
        // Data created by a   node
        String sender = "java -Node";
        String message = "Hello from the distributed system!";
       // Convert data to JSON format
       String jsonMessage = "{"
        + "\"sender\":\"" + sender + "\","
        + "\"message\":\"" + message + "\""
        + "}";
      // Send/display JSON message
      System.out.println("JSON Message:");
      System.out.println(jsonMessage);
    }
}
```

Output

```
JSON Message:
{"sender":"java -Node","message":"Hello from the distributed system!"}
BUILD SUCCESSFUL (total time: 0 seconds)
```

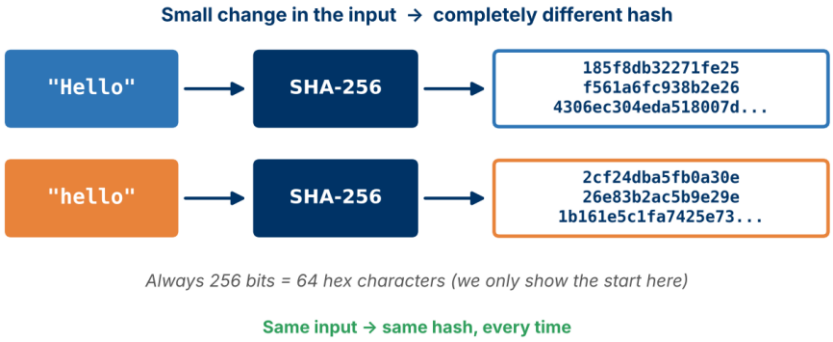
Problem 2: Node-B receives the report, but has the data changed?

Node-A sends the **important report** to **Node-B**. However, **during storage** or **transmission**, the **content** could **become corrupted or altered**.

Solution: SHA-256 (Secure Hash Algorithm)

What is a hash function?

A **hash function** takes any data (a word, a file, a movie) and gives back a short **fingerprint** of it. **SHA-256** gives a fingerprint of **256 bits**. We usually write it as **64 hexadecimal characters**.



Important facts

- **Fixed length:** the output is always 64 characters, even for a huge file.
- **Same input → same hash**, every time.
- **Tiny change → very different hash.** (`Hello` and `hello` look almost the same, but their hashes are completely different.)
- **Hashing is not encryption.** There is no key to "decrypt" a hash and get the original data back.

Where do we use it?

- **File integrity:** checking that a downloaded or received file was not changed.

program that demonstrates Security (Data Integrity) using the SHA 256 cryptographic hash algorithm. The sender creates a hash (digital fingerprint) of a message and sends both the message and the hash. The receiver recalculates the hash and compares it with the received hash to verify that the message was not altered during transmission.

Code

```
package integritydemo;
import  java.security.MessageDigest;
public class IntegrityDemo {
   // Method to generate SHA-256 hash
   public static String generateHash(String message) throws Exception {
      MessageDigest md = MessageDigest.getInstance("SHA-256");
      byte[] hashBytes = md.digest(message.getBytes());
      StringBuilder hash = new StringBuilder();
      for (byte b : hashBytes) {
         hash.append(String.format("%02x", b));
      }
        return hash.toString();
 }
   public static void main(String[] args) throws Exception {
       // Sender creates the message
       String message = "Transfer $100";
       // Sender generates the hash
       String sentHash = generateHash(message);
       System.out.println("Original Message: " + message);
       System.out.println("Sent Hash: " + sentHash);
       // Simulate receiving the message
       String receivedMessage = "Transfer $100";
       // Receiver recalculates the hash
       String receivedHash = generateHash(receivedMessage);
       // Verify integrity
       if (sentHash.equals(receivedHash)) {
          System.out.println("Integrity Check Passed: Message is unchanged.");
       } else {
          System.out.println("Integrity Check Failed: Message is changed.!");
       } } }
```

Output

```
Original Message: Transfer $100
Sent Hash: 1aab7a327cf10daee4c290ae2c96d6c321e082d318de8f30173910430de917e1
Integrity Check Passed: Message is unchanged.
BUILD SUCCESSFUL (total time: 0 seconds)
```

Problem 3: Node-B is too slow

The report reaches Node-B correctly, but the employee waits 10 seconds for a response.

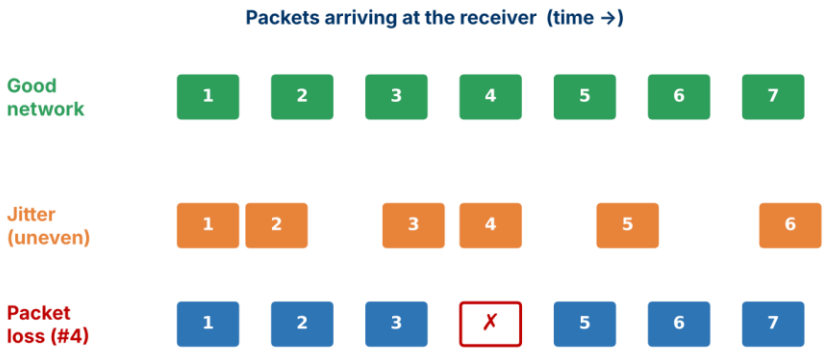
Solution use QoS (Quality of Service)

What is QoS?

QoS means **how good the network service is**. We measure it with numbers. These numbers decide how well an application works.

The four basic metrics

Metric	Meaning	Simple example
Latency	How long does a request take?	A message takes **100 ms**
Bandwidth	How much data can the network transfer per second?	A link supports **100 Mbps**
Jitter	How much does packet delay vary?	Voice packets arrive at **uneven** times
Packet loss	How many packets fail to arrive?	**2 out of 100** packets are lost



Different applications, different needs

Application	What matters most	Why
Video call	**Low latency** and **low jitter**	Voice and video must arrive quickly and smoothly
Large file transfer	**High bandwidth** and **complete delivery**	All the data must arrive; a few seconds more is fine

Networks can also give **priority** to important traffic. For example, a video call can be sent before a background download.

SLA: a promise about QoS

An **SLA** (**Service Level Agreement**) is a **promise** about the quality of a service.
Example: "Every request will be answered in **200 ms or less**."

To know if the promise is kept, we **measure** the QoS (for example the latency) and **compare** it with the SLA. This is called **QoS monitoring**.

Program that demonstrates Quality of Service (QoS) Monitoring using a Service Level Agreement (SLA).
The program simulates a network request by making the program wait for a certain amount of time, measures the latency, and then checks whether the response time meets the SLA requirement of 200 milliseconds or less.

Code

```
package qosmonitor;
public class QoSMonitor {
   public static void main(String[] args) throws InterruptedException {
       // SLA (maximum allowed latency)
       long SLA_LIMIT =200; // milliseconds
       // Record start time
       long startTime = System.currentTimeMillis();
       // Simulate processing a network request
       Thread.sleep(150);
       // Record end time
       long endTime = System.currentTimeMillis();
       // Calculate latency
       long latency = endTime - startTime;
       System.out.println("Request Latency: " + latency + " ms");
       // Check if SLA is met
       if (latency <= SLA_LIMIT) {
        System.out.println("SLA Met: Service is performing correctly.");
       }else {
        System.out.println("SLA Violated: Request took too long.");
       }
   }
}
```

Output

```
run:
Request Latency: 177 ms
SLA Met: Service is performing correctly.
BUILD SUCCESSFUL (total time: 0 seconds)
```

Problem 4: Node-A doesn't know how to request the service

Node-A knows that **Node-B provides printing**, but there is **another problem**.

Should Node-A **call printFile() or printDocument()? What information must it send? What should Node-B return? What happens if printing fails?**

Without agreed rules, each developer may implement the service differently.

Solution Interfaces in Distributed Systems

What is an interface?

An **interface** is a **contract**. It lists **what a component can do** and **how to ask it**, but it does **not** show **how** the work is done inside.

Real-life example: the buttons of a coffee machine are its interface. You press "Espresso" and get a coffee. You do not need to know how the machine works inside.

Program: Openness and Heterogeneity Using Interfaces

This program demonstrates Openness and Heterogeneity in a distributed system. The application interacts with a **common interface (Storage)** rather than a specific storage technology. As a result, the storage implementation can be changed from **local storage** to **cloud storage** without modifying the main application logic.

Code

```
package opennessdemo;
// Common interface
interface Storage {
  void save(String data);
}
// Local storage implementation
class LocalStorage implements Storage {
    public void save(String data) {
      System.out.println("Saved locally: " + data);
    }
}
// Cloud storage implementation
class CloudStorage implements Storage {
    public void save(String data) {
      System.out.println("Saved to cloud: " + data);
    }
}
public class OpennessDemo {
    public static void main(String[] args) {
        Storage storageC = new CloudStorage();
        // Save a file
        storageC.save("lecture-notes.txt");
        Storage storageL = new LocalStorage();
        // Save a file
        storageL.save("Section-notes.txt");
    }

}
```

Output

```
run:
Saved to cloud: lecture-notes.txt
Saved locally: Section-notes.txt
BUILD SUCCESSFUL (total time: 0 seconds)
```

Problem 5: Node-B suddenly stops responding

Everything is working. Node-A sends requests, Node-B prints the reports, and the response time is acceptable.
Then Node-B stops sending responses.
Node-A doesn't know why:

- Has Node-B crashed?
- Is the network disconnected?
- Is Node-B just slow?

Node-A needs a way to monitor Node-B.

Solution: Heartbeat

What is a heartbeat?

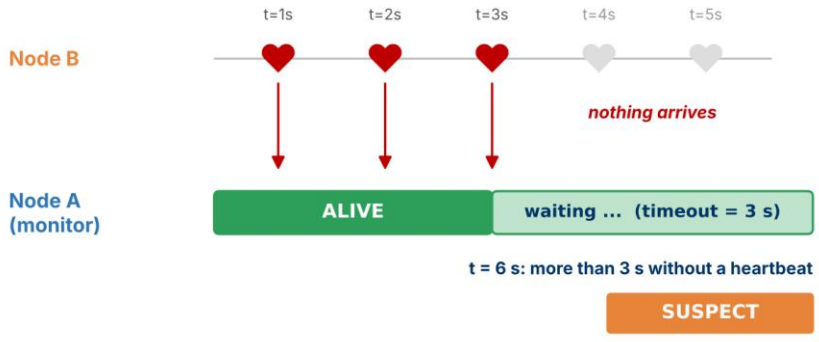
A **heartbeat** is a **small message sent again and again** to say: **"I am still alive!"** Like a doctor checking your heartbeat, a node checks that another node is still responding.

Why do we need it? In a distributed system, a node can fail at any time. We must **find out quickly**, so the system can react.

Intervals and timeouts

- **Heartbeat interval:** how often the heartbeat is sent (for example, every **1 second**).
- **Timeout:** how long the monitor waits before it worries (for example, **3 seconds**).
- If no heartbeat arrives before the timeout, the monitor marks the node as suspected.

Example: Node-A monitors Node-B. Node-B sends a heartbeat every **1 second**, then stops after the third one. Node-A waits **3 seconds** without any heartbeat, so it marks Node-B as **SUSPECT**.



Node states

State	Meaning
`ALIVE`	Heartbeats are arriving on time
`SUSPECT`	A heartbeat is late or missing — maybe there is a problem
`UNREACHABLE`	No heartbeat for a long time — we treat the node as not available

What happens after a suspected failure?

- Send an **alert** to the administrator.
- **Retry** the request.
- **Select another node** to do the work.

> **Be careful**
>
> A missing heartbeat **does not prove** that Node-B has crashed. The **network may be slow**, or the heartbeat message may have been **lost**. That is why we say **suspect**, not **dead**.

Program: Failure Detection Using a Heartbeat Mechanism

This program demonstrates Failure Detection and Independent Failures in a distributed system. A monitoring process **records the time of the last heartbeat received** from a node. If no heartbeat is received within **3 seconds**, the monitor **suspects** that the node has crashed or become unreachable.

Code

```
package heartbeatdemo;
public class HeartbeatDemo {
   // Time when the last heartbeat was received
   private static long lastHeartbeat = System.currentTimeMillis();
   public static void main(String[] args) {
      // Simulated node sending heartbeats
      Thread node = new Thread(() -> {
         try {
          // Send 3 heartbeats, then stop (simulate failure)
          for (int i = 1; i <= 3; i++) {
          Thread.sleep(1000);
          lastHeartbeat = System.currentTimeMillis();
          System.out.println("Heartbeat received");
          }
          System.out.println("Node has stopped sending heartbeats.");
         }
         catch (InterruptedException e) {
            e.printStackTrace();
         } });
      // Monitor thread
      Thread monitor = new Thread(() -> {
         while (true) {
            long currentTime = System.currentTimeMillis();
            if (currentTime - lastHeartbeat >3000) {
               System.out.println(  "ALERT: Node failure suspected!");
               break;
            } } });
      node.start();
      monitor.start();
   }  }
```

Output

```
Heartbeat received
Heartbeat received
Heartbeat received
Node has stopped sending heartbeats.
ALERT: Node failure suspected!
BUILD SUCCESSFUL (total time: 6 seconds)
```